```
# ================================================================================
# COMPLETE RAIL TEMPERATURE PINN - SAME INPUT AS BEFORE
# ================================================================================

# ================================================================================
# CELL 0: INSTALL DEPENDENCIES
# ================================================================================

!pip install torch torchvision torchaudio -q
!pip install requests pandas numpy matplotlib seaborn -q

print("✅ Dependencies installed!")

# ================================================================================
# CELL 1: IMPORTS
# ================================================================================

import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import requests
import time
import warnings
warnings.filterwarnings('ignore')

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"✅ Using device: {device}")

# ================================================================================
# CELL 2: CONFIGURATION
# ================================================================================

FORTYGUARD_API_KEY = "YOUR_FORTYGUARD_API_KEY_HERE"
FORTYGUARD_BASE_URL = "https://api.fortyguard.com/v1"

# ================================================================================
# CELL 3: RAIL POINTS
# ================================================================================

RAIL_POINTS = [
    {"id": 1, "name": "New York Penn Station", "lat": 40.7502, "lon": -73.9928
    {"id": 2, "name": "Portal Bridge", "lat": 40.7536, "lon": -74.0947, "type"
    {"id": 3, "name": "Newark Penn Station", "lat": 40.7347, "lon": -74.1642, '
    {"id": 4, "name": "New Brunswick Station", "lat": 40.4964, "lon": -74.4464
    {"id": 5, "name": "Elizabeth Station", "lat": 40.6668, "lon": -74.2150, "ty
    {"id": 6, "name": "Linden Yard", "lat": 40.6393, "lon": -74.2290, "type": '
    {"id": 7, "name": "Princeton Junction", "lat": 40.3167, "lon": -74.6233, "
    {"id": 8, "name": "Trenton Transit Center", "lat": 40.2189, "lon": -74.754
    {"id": 9, "name": "Philadelphia Area", "lat": 39.9500, "lon": -75.1500, "ty
    {"id": 10, "name": "Baltimore Penn Station", "lat": 39.3075, "lon": -76.615
    {"id": 11, "name": "Aberdeen Station", "lat": 39.5084, "lon": -76.1632, "ty
    {"id": 12, "name": "Washington Union Station", "lat": 38.8972, "lon": -77.(
]

# ================================================================================
```

```
# CELL 4: API FUNCTIONS (SAME AS BEFORE)
# ================================================================================

def poll_status(activity_id, max_attempts=20, delay=3):
    status_url = f"{FORTYGUARD_BASE_URL}/status/{activity_id}"

    for attempt in range(max_attempts):
        try:
            response = requests.get(status_url, headers={'api-key': FORTYGUARD_

            if response.status_code == 200:
                data = response.json()
                status = data.get('status') or data.get('data', {}).get('status

                if status == 'Completed':
                    return data.get('result') or data.get('data', {}).get('resu
                elif status == 'Failed':
                    return None
                else:
                    time.sleep(delay)
            else:
                time.sleep(delay)
        except:
            time.sleep(delay)

    return None

def get_fortyguard_temperature(lat, lon, date_str, hour):
    time_str = f"{hour:02d}:00"

    polygon = [
        [lon - 0.005, lat - 0.005],
        [lon + 0.005, lat - 0.005],
        [lon + 0.005, lat + 0.005],
        [lon - 0.005, lat + 0.005],
        [lon - 0.005, lat - 0.005]
    ]

    response = requests.post(
        f'{FORTYGUARD_BASE_URL}/heatmap',
        headers={'api-key': FORTYGUARD_API_KEY},
        json={
            'polygon_aoi': {
                'type': 'FeatureCollection',
                'features': [{
                    'type': 'Feature',
                    'properties': {},
                    'geometry': {
                        'type': 'Polygon',
                        'coordinates': [polygon]
                    }
                }]
            },
            'date_time': {
                'start_date': date_str,
                'start_time': time_str,
                'filter_type': 1
            },
            'granularity': 100
        }
```

```
        )

        if response.status_code != 200:
            return None

        data = response.json()
        activity_id = data.get('data', {}).get('activity_id')

        if not activity_id:
            return None

        result = poll_status(activity_id)

        if not result:
            return None

        map_data = result.get('map_data', {})
        features = map_data.get('features', [])

        for feature in features:
            props = feature.get('properties', {})
            temp = props.get('average_temperature') or props.get('temperature') or
            if temp is not None:
                if isinstance(temp, list):
                    return float(temp[0]) if len(temp) > 0 else None
                return float(temp)

        return None

    def get_fortyguard_environmental(lat, lon, temperature, date_str, hour):
        time_str = f"{hour:02d}:00"

        response = requests.post(
            f'{FORTYGUARD_BASE_URL}/env_params',
            headers={'api-key': FORTYGUARD_API_KEY},
            json={
                'latitude': lat,
                'longitude': lon,
                'temperature': temperature,
                'date_time': {
                    'start_date': date_str,
                    'start_time': time_str,
                    'filter_type': 1
                }
            }
        )

        if response.status_code != 200:
            return None

        data = response.json()
        activity_id = data.get('data', {}).get('activity_id')

        if not activity_id:
            return None

        result = poll_status(activity_id)

        if not result:
            return None
```

```
        locations = result.get('locations', [])

        if not locations:
            return None

        loc = locations[0]
        params = loc.get('parameters', {})
        solar = loc.get('solar_irradiance', {})
        clear_sky = solar.get('clear_sky', {})

        def extract_value(data):
            if data is None:
                return None
            if isinstance(data, list):
                return data[0] if len(data) > 0 else None
            return data

        return {
            'humidity_pct': extract_value(params.get('relative_humidity_percent')),
            'cloud_cover_octas': extract_value(params.get('cloud_cover_octas')),
            'ghi_wm2': extract_value(clear_sky.get('ghi')),
            'dni_wm2': extract_value(clear_sky.get('dni')),
            'dhi_wm2': extract_value(clear_sky.get('dhi')),
            'co2_ppm': extract_value(params.get('co2_ppm')),
            'elevation_m': extract_value(loc.get('elevation'))
        }

    def get_openmeteo_wind_speed(lat, lon, date_str, hour):
        url = "https://archive-api.open-meteo.com/v1/archive"

        params = {
            "latitude": lat,
            "longitude": lon,
            "start_date": date_str,
            "end_date": date_str,
            "hourly": ["wind_speed_10m", "soil_temperature_0_to_7cm"],
            "timezone": "America/New_York",
            "format": "json"
        }

        try:
            response = requests.get(url, params=params, timeout=30)

            if response.status_code == 200:
                data = response.json()
                hourly_data = data.get('hourly', {})

                wind_speeds = hourly_data.get('wind_speed_10m', [])
                soil_temps = hourly_data.get('soil_temperature_0_to_7cm', [])

                wind = wind_speeds[hour] if wind_speeds and len(wind_speeds) > hou
                soil_temp = soil_temps[hour] if soil_temps and len(soil_temps) > h

                return {'wind_speed': wind, 'soil_temp': soil_temp}
            else:
                return None
        except:
            return None
```

```
# ============================================================================
# CELL 5: PHYSICS MODEL (Includes Soil Temperature, Solar, Wind, etc.)
# ============================================================================

def calculate_physics_rail_temperature(air_temp, wind_speed, solar_radiation, h
                                       cloud_cover, soil_temp, hour, elevation
    """
    Physics-based rail temperature model.
    Includes: Solar radiation, wind cooling, ground (soil) temperature,
              humidity, cloud cover, elevation, diurnal cycle.
    """
    # Solar absorption coefficient for steel rail
    solar_absorptivity = 0.7

    # Cloud cover reduces solar radiation (0-8 octas → 0-1 factor)
    cloud_factor = 1 - (cloud_cover / 8) * 0.5

    # Effective solar radiation reaching the rail
    effective_solar = solar_radiation * cloud_factor

    # Solar heating
    solar_heating = solar_absorptivity * effective_solar / 100

    # Convective heat transfer coefficient (depends on wind speed)
    h_convection = 5.0 + 8.0 * wind_speed

    # Sky temperature (simplified with humidity effect)
    sky_temp = air_temp - 5 - 0.1 * humidity

    # Ground (soil) temperature effect
    ground_effect = 0.1 * (soil_temp - air_temp)

    # Convective cooling
    convective_cooling = h_convection * (air_temp - sky_temp) / 100

    # Radiative cooling
    epsilon = 0.85
    sigma = 5.67e-8
    radiative_cooling = epsilon * sigma * ((air_temp + 273.15)**4 - (sky_temp +

    # Elevation effect (temperature decreases with altitude)
    elevation_effect = -0.006 * elevation

    # Rail temperature = air temp + solar heating - cooling + ground effect
    rail_temp = air_temp + solar_heating - convective_cooling * 0.4 - radiative

    # Add diurnal variation (peak at 14:00-16:00)
    diurnal_amplitude = 5.0
    diurnal_offset = diurnal_amplitude * np.sin(np.pi * (hour - 14) / 12)
    rail_temp += diurnal_offset

    # Add ground temperature effect
    rail_temp += ground_effect * 0.2

    # Add elevation effect
    rail_temp += elevation_effect

    # Humidity effect on cooling
    humidity_factor = 1 + 0.005 * (humidity - 50)
    rail_temp = air_temp + (rail_temp - air_temp) / humidity_factor
```

```
        return rail_temp

    # ============================================================================
    # CELL 6: PINN MODEL
    # ============================================================================

    class RailTempPINN(nn.Module):
        """
        Physics-Informed Neural Network for Rail Temperature Prediction.
        """

        def __init__(self, input_dim=8, hidden_layers=[256, 256, 128, 64]):
            super(RailTempPINN, self).__init__()

            layers = []
            layers.append(nn.Linear(input_dim, hidden_layers[0]))
            layers.append(nn.Tanh())

            for i in range(len(hidden_layers) - 1):
                layers.append(nn.Linear(hidden_layers[i], hidden_layers[i+1]))
                layers.append(nn.Tanh())

            layers.append(nn.Linear(hidden_layers[-1], 1))

            self.net = nn.Sequential(*layers)

            # Physics constants
            self.solar_absorptivity = 0.7
            self.epsilon = 0.85
            self.sigma = 5.67e-8

            self._initialize_weights()

        def _initialize_weights(self):
            for m in self.net.modules():
                if isinstance(m, nn.Linear):
                    nn.init.xavier_normal_(m.weight)
                    nn.init.zeros_(m.bias)

        def forward(self, x):
            return self.net(x)

        def physics_loss(self, x, rail_temp_pred):
            """
            Physics-informed loss function.
            """
            # Extract normalized inputs
            hour = x[:, 0:1]
            air_temp = x[:, 1:2] * 50
            wind_speed = x[:, 2:3] * 20
            solar_radiation = x[:, 3:4] * 1000
            humidity = x[:, 4:5] * 100
            cloud_cover = x[:, 5:6] * 8
            soil_temp = x[:, 6:7] * 50
            elevation = x[:, 7:8] * 1000

            # Denormalize predicted rail temperature
            rail_temp = rail_temp_pred * 50
```

```
        # Physics constraint 1: Energy balance
        cloud_factor = 1 - cloud_cover / 8
        solar_heating = self.solar_absorptivity * solar_radiation * cloud_facto

        h_convection = 5.0 + 8.0 * wind_speed
        sky_temp = air_temp - 5 - 0.1 * humidity
        convective_cooling = h_convection * (air_temp - sky_temp) / 100

        radiative_cooling = self.epsilon * self.sigma * (
            (air_temp + 273.15)**4 - (sky_temp + 273.15)**4
        ) / 10

        ground_effect = 0.1 * (soil_temp - air_temp)

        # Physics-based rail temperature
        rail_temp_physics = air_temp + solar_heating - convective_cooling * 0.4

        # Physics loss
        physics_loss = torch.mean((rail_temp - rail_temp_physics) ** 2)

        # Smoothness loss (diurnal cycle should be smooth)
        if x.shape[0] > 2:
            temp_diff = rail_temp[1:] - rail_temp[:-1]
            smoothness_loss = torch.mean((temp_diff[1:] - temp_diff[:-1]) ** 2)
        else:
            smoothness_loss = torch.tensor(0.0, device=x.device)

        # Bound loss (physical limits)
        lower_bound = -20
        upper_bound = 80
        bound_loss = torch.mean(torch.relu(lower_bound - rail_temp)) + torch.me

        total_physics_loss = physics_loss + smoothness_loss + bound_loss * 0.01

        return total_physics_loss

    def combined_loss(self, x, rail_temp_pred, rail_temp_target=None):
        data_loss = 0.0
        if rail_temp_target is not None:
            data_loss = torch.mean((rail_temp_pred - rail_temp_target) ** 2)

        phys_loss = self.physics_loss(x, rail_temp_pred)

        lambda_phys = 0.1
        total_loss = data_loss + lambda_phys * phys_loss

        return total_loss, data_loss, phys_loss

# ============================================================================
# CELL 7: TRAIN PINN ON 24-HOUR DATA
# ============================================================================

def train_pinn_on_24hour_data(air_temp, wind_speed, solar_radiation, humidity,
                              cloud_cover, soil_temp, elevation, epochs=5000):
    """
    Generate 24-hour training data and train PINN.
    """
    print("\n📊 Generating 24-hour training data...")

    # Generate 24-hour training data using physics model
```

```
        training_data = []
        for hour in range(24):
            rail_temp = calculate_physics_rail_temperature(
                air_temp, wind_speed, solar_radiation, humidity,
                cloud_cover, soil_temp, hour, elevation
            )

            training_data.append({
                'hour': hour / 24.0,
                'air_temp': air_temp / 50.0,
                'wind_speed': wind_speed / 20.0,
                'solar_radiation': solar_radiation / 1000.0,
                'humidity': humidity / 100.0,
                'cloud_cover': cloud_cover / 8.0,
                'soil_temp': soil_temp / 50.0,
                'elevation': elevation / 1000.0,
                'rail_temp': rail_temp / 50.0
            })

        # Prepare data for PINN
        X = []
        y = []

        for record in training_data:
            X.append([
                record['hour'],
                record['air_temp'],
                record['wind_speed'],
                record['solar_radiation'],
                record['humidity'],
                record['cloud_cover'],
                record['soil_temp'],
                record['elevation']
            ])
            y.append(record['rail_temp'])

        X = torch.tensor(np.array(X), dtype=torch.float32)
        y = torch.tensor(np.array(y).reshape(-1, 1), dtype=torch.float32)

        print(f"   ✅ Generated {len(training_data)} training samples (24 hours)")

        # Create and train PINN
        print("\n🚀 Training PINN...")
        print("-" * 50)

        model = RailTempPINN(input_dim=8, hidden_layers=[256, 256, 128, 64]).to(dev
        optimizer = optim.Adam(model.parameters(), lr=1e-3)
        scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=500, f

        history = {'total_loss': [], 'data_loss': [], 'phys_loss': []}

        for epoch in range(epochs):
            rail_temp_pred = model(X)

            total_loss, data_loss, phys_loss = model.combined_loss(X, rail_temp_pre

            optimizer.zero_grad()
            total_loss.backward()
            optimizer.step()
            scheduler.step(total_loss)
```

```
        history['total_loss'].append(total_loss.item())
        history['data_loss'].append(data_loss.item())
        history['phys_loss'].append(phys_loss.item())

        if epoch % 1000 == 0:
            print(f"Epoch {epoch:4d}: Total={total_loss.item():.4f}, "
                  f"Data={data_loss.item():.4f}, Physics={phys_loss.item():.4f}

    return model, history, training_data

def predict_with_pinn(model, hour, air_temp, wind_speed, solar_radiation,
                      humidity, cloud_cover, soil_temp, elevation):
    """
    Predict rail temperature using trained PINN.
    """
    # Normalize inputs
    hour_norm = hour / 24.0
    air_temp_norm = air_temp / 50.0
    wind_speed_norm = wind_speed / 20.0
    solar_radiation_norm = solar_radiation / 1000.0
    humidity_norm = humidity / 100.0
    cloud_cover_norm = cloud_cover / 8.0
    soil_temp_norm = soil_temp / 50.0
    elevation_norm = elevation / 1000.0

    X = np.array([[hour_norm, air_temp_norm, wind_speed_norm, solar_radiation_n
                   humidity_norm, cloud_cover_norm, soil_temp_norm, elevation_n
    X = torch.tensor(X, dtype=torch.float32)

    with torch.no_grad():
        rail_temp_pred = model(X).numpy()[0][0] * 50.0

    return rail_temp_pred

# ============================================================================
# CELL 8: MAIN EXECUTION (SAME INPUT AS BEFORE)
# ============================================================================

print("\n" + "=" * 70)
print("🚀 RAIL THERMAL STRESS PREDICTOR (WITH PINN)")
print("=" * 70)

# Display available points
print("\n📍 Available Points:")
for p in RAIL_POINTS:
    print(f"   {p['id']:2d}. {p['name']}")

# Get user input
print("\n" + "-" * 40)
try:
    point_id = int(input("Enter point ID (1-12): "))
    date_str = input("Enter date (YYYY-MM-DD): ")
    hour = int(input("Enter hour (0-23): "))
except ValueError:
    print("❌ Invalid input. Using defaults.")
    point_id = 1
    date_str = "2026-08-01"
    hour = 14
```

```
point = next((p for p in RAIL_POINTS if p['id'] == point_id), RAIL_POINTS[0])

print(f"\n📍 Location: {point['name']}")
print(f"📅 Date: {date_str}, Hour: {hour:02d}:00")

# ============================================================================
# COLLECT DATA
# ============================================================================

print("\n📊 Collecting data from APIs...")

# Get temperature
air_temp = get_fortyguard_temperature(point['lat'], point['lon'], date_str, hou
if air_temp is None:
    air_temp = 27.0
print(f"   ✅ Air Temperature: {air_temp:.1f}°C")

# Get environmental parameters
env_data = get_fortyguard_environmental(point['lat'], point['lon'], air_temp, 
if env_data is None:
    env_data = {'humidity_pct': 60.0, 'cloud_cover_octas': 4.0, 'ghi_wm2': 400

humidity = env_data.get('humidity_pct', 60.0)
cloud_cover = env_data.get('cloud_cover_octas', 4.0)
solar_radiation = env_data.get('ghi_wm2', 400.0)
elevation = env_data.get('elevation_m', 10.0)

print(f"   ✅ Humidity: {humidity:.1f}%")
print(f"   ✅ Solar Radiation: {solar_radiation:.1f} W/m²")
print(f"   ✅ Cloud Cover: {cloud_cover:.1f} octas")

# Get wind speed and soil temperature
wind_data = get_openmeteo_wind_speed(point['lat'], point['lon'], date_str, hou
if wind_data is None:
    wind_speed = 5.0
    soil_temp = air_temp - 2.0
else:
    wind_speed = wind_data.get('wind_speed', 5.0)
    soil_temp = wind_data.get('soil_temp', air_temp - 2.0)

print(f"   ✅ Wind Speed: {wind_speed:.1f} m/s")
print(f"   ✅ Soil Temperature: {soil_temp:.1f}°C")

# ============================================================================
# TRAIN PINN ON 24-HOUR DATA
# ============================================================================

model, history, training_data = train_pinn_on_24hour_data(
    air_temp, wind_speed, solar_radiation, humidity,
    cloud_cover, soil_temp, elevation, epochs=5000
)

# ============================================================================
# PREDICT FOR YOUR HOUR
# ============================================================================

print("\n📊 PINN Prediction...")
pinn_rail_temp = predict_with_pinn(
    model, hour, air_temp, wind_speed, solar_radiation,
    humidity, cloud_cover, soil_temp, elevation
```

```
)
print(f"   ✅ PINN Rail Temperature: {pinn_rail_temp:.1f}°C")

# Physics-based for comparison
physics_rail_temp = calculate_physics_rail_temperature(
    air_temp, wind_speed, solar_radiation, humidity,
    cloud_cover, soil_temp, hour, elevation
)
print(f"   ✅ Physics-based Rail Temp: {physics_rail_temp:.1f}°C")

# ============================================================================
# CALCULATE THERMAL STRESS
# ============================================================================

print("\n📊 Thermal Stress Calculation...")
E = 210000  # MPa
alpha = 1.1e-5
T_ref = 27.0

thermal_stress = E * alpha * (pinn_rail_temp - T_ref)
critical_stress = 200.0

if thermal_stress < 50:
    risk_level = "🟢 Low"
    risk_color = "green"
elif thermal_stress < 100:
    risk_level = "🟡 Medium"
    risk_color = "yellow"
elif thermal_stress < 150:
    risk_level = "🟠 High"
    risk_color = "orange"
else:
    risk_level = "🔴 Critical"
    risk_color = "red"

print(f"   ✅ Thermal Stress: {thermal_stress:.1f} MPa")
print(f"   ✅ Risk Level: {risk_level}")

# ============================================================================
# VISUALIZATION
# ============================================================================

print("\n📊 Generating visualization...")

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Plot 1: 24-Hour Temperature Comparison
ax1 = axes[0, 0]
hours = np.arange(24)
phys_temps = []
pinn_temps = []

for h in hours:
    phys_temps.append(calculate_physics_rail_temperature(
        air_temp, wind_speed, solar_radiation, humidity,
        cloud_cover, soil_temp, h, elevation
    ))
    pinn_temps.append(predict_with_pinn(
        model, h, air_temp, wind_speed, solar_radiation,
        humidity, cloud_cover, soil_temp, elevation
```

```
    ))

ax1.plot(hours, phys_temps, 'b-', linewidth=2, label='Physics-based')
ax1.plot(hours, pinn_temps, 'r--', linewidth=2, label='PINN Prediction')
ax1.scatter([hour], [physics_rail_temp], color='blue', s=100, zorder=5, label=
ax1.scatter([hour], [pinn_rail_temp], color='red', s=100, zorder=5, label='PINN
ax1.axvline(x=hour, color='gray', linestyle=':', alpha=0.5)
ax1.set_xlabel('Hour of Day')
ax1.set_ylabel('Rail Temperature (°C)')
ax1.set_title(f'{point["name"]} - Rail Temperature Prediction')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Plot 2: Training Loss
ax2 = axes[0, 1]
ax2.plot(history['total_loss'], label='Total Loss')
ax2.plot(history['data_loss'], label='Data Loss')
ax2.plot(history['phys_loss'], label='Physics Loss')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Loss')
ax2.set_title('PINN Training Loss')
ax2.legend()
ax2.grid(True, alpha=0.3)
ax2.set_yscale('log')

# Plot 3: Thermal Stress Gauge
ax3 = fig.add_subplot(2, 2, 3, projection='polar')
ax3.set_theta_zero_location('N')
ax3.set_theta_direction(-1)
ax3.set_ylim(0, 1)

thresholds = [0, 60, 100, 150, 200]
colors_gauge = ['#2ecc71', '#f1c40f', '#e67e22', '#e74c3c']

for i in range(len(thresholds) - 1):
    start = thresholds[i] / 200 * np.pi
    end = thresholds[i+1] / 200 * np.pi
    ax3.barh(0.5, end - start, left=start, height=0.3,
            color=colors_gauge[i], alpha=0.7, edgecolor='black')

stress_clamped = min(max(thermal_stress, 0), 200)
angle = stress_clamped / 200 * np.pi
ax3.plot([angle, angle], [0.2, 0.8], 'k-', linewidth=3)

ax3.text(np.pi/2, 0.15, '0 MPa', ha='center', fontsize=10)
ax3.text(0, 0.15, '200 MPa', ha='center', fontsize=10)
ax3.text(angle, 0.9, f'{thermal_stress:.1f} MPa', ha='center', fontsize=12, fo
ax3.text(np.pi/2, 1.1, f'Risk: {risk_level}', ha='center', fontsize=14, fontwe
ax3.set_title('Thermal Stress Gauge')

# Plot 4: Summary
ax4 = axes[1, 1]
ax4.axis('off')

summary_text = f"""
📊 THERMAL STRESS SUMMARY

📍 Location: {point['name']}
📅 Date: {date_str}
🕐 Hour: {hour:02d}:00
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
🌡️ Air Temperature:     {air_temp:.1f}°C
🛤️ Physics Rail Temp:    {physics_rail_temp:.1f}°C
🧠 PINN Rail Temp:       {pinn_rail_temp:.1f}°C
💨 Wind Speed:          {wind_speed:.1f} m/s
💧 Humidity:            {humidity:.0f}%
☀️ Solar Radiation:      {solar_radiation:.0f} W/m²
🌍 Soil Temperature:     {soil_temp:.1f}°C

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
⚡ Thermal Stress:       {thermal_stress:.1f} MPa
📊 Risk Level:          {risk_level}
"""

ax4.text(0.1, 0.95, summary_text, transform=ax4.transAxes,
        fontsize=12, verticalalignment='top', fontfamily='monospace',
        bbox=dict(boxstyle='round', facecolor='#f0f0f0', alpha=0.8))

plt.tight_layout()
plt.savefig('pinn_rail_temp_prediction.png', dpi=150, bbox_inches='tight')
plt.show()

print("\n✅ Visualization saved to 'pinn_rail_temp_prediction.png'")

# ============================================================================
# EXPORT RESULTS
# ============================================================================

results_df = pd.DataFrame([{
    'point_id': point['id'],
    'point_name': point['name'],
    'date': date_str,
    'hour': hour,
    'air_temperature_c': air_temp,
    'physics_rail_temp_c': physics_rail_temp,
    'pinn_rail_temp_c': pinn_rail_temp,
    'wind_speed_ms': wind_speed,
    'humidity_pct': humidity,
    'solar_radiation_wm2': solar_radiation,
    'soil_temperature_c': soil_temp,
    'thermal_stress_mpa': thermal_stress,
    'risk_level': risk_level
}])

results_df.to_csv('pinn_rail_temp_results.csv', index=False)
print("\n✅ Results saved to 'pinn_rail_temp_results.csv'")

print("\n" + "=" * 70)
print("✅ PREDICTION COMPLETE!")
print("=" * 70)
```

```
✅ Dependencies installed!
✅ Using device: cpu

======================================================================
🚀 RAIL THERMAL STRESS PREDICTOR (WITH PINN)
======================================================================

📍 Available Points:
   1. New York Penn Station
   2. Portal Bridge
   3. Newark Penn Station
   4. New Brunswick Station
   5. Elizabeth Station
   6. Linden Yard
   7. Princeton Junction
   8. Trenton Transit Center
   9. Philadelphia Area
  10. Baltimore Penn Station
  11. Aberdeen Station
  12. Washington Union Station
```